

1. DESCRIPCIÓN DEL SISTEMA DE INFORMACIÓN PARKNOW

ParkNow es un sistema de información web diseñado para automatizar y optimizar el control de acceso vehicular en parqueaderos públicos. La solución permitirá registrar de manera ágil la entrada y salida de vehículos, gestionar la disponibilidad de espacios en tiempo real, generar reportes administrativos y operativos, así como administrar usuarios controladores con diferentes niveles de acceso.

El sistema está orientado a empresas de parqueaderos que buscan modernizar sus operaciones, reducir errores derivados de procesos manuales y mejorar la eficiencia en la gestión diaria. En su primera fase se enfocará en la digitalización de los procesos básicos, sin incluir aún tecnologías de sensores IoT, pero con la visión de convertirse en una plataforma escalable y adaptable a futuras necesidades.

Además, **ParkNow** ofrecerá mecanismos de alertas cuando el número de espacios disponibles llegue a un nivel crítico, y brindará reportes detallados por fecha, tipo de vehículo y usuario controlador, lo cual facilitará la toma de decisiones gerenciales.

En esta etapa inicial, el sistema se implementará para una única sede de parqueadero en la ciudad de Tunja, con la proyección de escalar a múltiples sedes en versiones futuras. Asimismo, se deja abierta la posibilidad de integrar nuevas funcionalidades como sensores inteligentes, suscripciones y pagos electrónicos en fases posteriores.

2. REQUERIMIENTOS DE HARDWARE Y SOFTWARE

2.1 Requerimientos Mínimos de Hardware

Para la correcta implementación y funcionamiento del sistema de información ParkNow, se establecen los siguientes requerimientos mínimos de hardware, diferenciados entre el servidor de aplicación, las estaciones de trabajo de los controladores y la infraestructura de red:

➤ Servidor de Aplicación

El servidor donde se desplegarán la aplicación web, la API backend y la base de datos debe cumplir con las siguientes especificaciones mínimas:

- Procesador (CPU): Intel Core i5 de 8.^a generación o AMD Ryzen 5 equivalente (mínimo 4 núcleos).
- Memoria RAM: 8 GB DDR4 (se recomienda 16 GB para mayor rendimiento).
- Almacenamiento: Unidad SSD de 500 GB (mínimo 250 GB disponibles).
- Conectividad de Red: Tarjeta de red Gigabit Ethernet.
- Puertos de Expansión: Al menos 2 puertos USB 3.0.

➤ **Estaciones de Trabajo (Controladores)**

Los equipos de los controladores que utilizarán el sistema a través del navegador web deben contar con las siguientes características mínimas:

- Procesador (CPU): Intel Core i3 de 7.^a generación o AMD Ryzen 3 equivalente.
- Memoria RAM: 4 GB DDR4 (se recomienda 8 GB).
- Almacenamiento: Unidad HDD o SSD de al menos 250 GB.
- Pantalla: Monitor de 19 pulgadas como mínimo, con resolución de 1366x768 píxeles.
- Conectividad de Red: Conexión Ethernet o WiFi 802.11n.

➤ **Infraestructura de Red**

Para garantizar la disponibilidad del sistema y la conectividad entre los diferentes nodos se requiere la siguiente infraestructura mínima:

- Router/Switch: Capacidad de soportar al menos 10 dispositivos concurrentes.
- Ancho de Banda: Conexión a internet de mínimo 50 Mbps.
- Sistema de Energía: UPS de 1500VA para respaldo de energía en el servidor y los equipos de red.

2.2. Requerimientos Mínimos de Software

Los componentes de software necesarios para la operación del sistema se dividen en tres grupos: sistema operativo, software base y herramientas de desarrollo.

➤ Sistema Operativo del Servidor

El sistema puede implementarse sobre cualquiera de las siguientes plataformas:

- Ubuntu Server 20.04 LTS o superior.
- Windows Server 2019 o superior.
- CentOS 8 o Red Hat Enterprise Linux 8.

➤ Software Base

Los servicios que soportarán la aplicación deben cumplir con las siguientes versiones mínimas:

- Node.js: versión 18.0 o superior.
- PostgreSQL: versión 13.0 o superior.
- Nginx: versión 1.18 o superior.

➤ Navegadores Web Soportados

El sistema podrá ejecutarse en los siguientes navegadores actualizados:

- Google Chrome versión 90 o superior.
- Mozilla Firefox versión 88 o superior.
- Microsoft Edge versión 90 o superior.
- Safari versión 14 o superior.

➤ Herramientas de Desarrollo

Para el desarrollo, mantenimiento y despliegue del sistema se contemplan las siguientes herramientas:

- Git: versión 2.30 o superior.
- npm: versión 8.0 o superior.
- Docker: versión 20.10 o superior (opcional para despliegues en contenedores).

3. ARQUITECTURA LÓGICA

3.1 Descripción de la Arquitectura Lógica

El sistema de información ParkNow adopta una arquitectura de tres capas (*3-tier architecture*), reconocida como una de las más utilizadas en el desarrollo de sistemas distribuidos debido a su claridad en la separación de responsabilidades, su escalabilidad y facilidad de mantenimiento. La definición de estas capas busca garantizar que los cambios en una de ellas no afecten directamente a las demás, permitiendo la evolución del sistema de manera modular.

- **Capa de Presentación (Frontend):** Esta capa corresponde a la interfaz con la cual interactúan los usuarios finales, en este caso, los controladores y administradores del parqueadero. Está desarrollada en React, lo que permite crear una experiencia de usuario dinámica, intuitiva y adaptable a diferentes dispositivos. Entre sus principales responsabilidades se encuentran:
 - La validación inicial de formularios y datos ingresados.
 - La gestión de sesiones de usuario en el cliente.
 - La comunicación con la capa de aplicación mediante solicitudes HTTP/HTTPS a la API REST.
 - La representación en tiempo real del estado de ocupación de los parqueaderos.
Esta capa asegura que el usuario final no necesite conocer los detalles técnicos del sistema, sino únicamente interactuar con una interfaz amigable y funcional.

- **Capa de Aplicación / Lógica de Negocio (Backend):** Desarrollada en Node.js con Express, constituye el núcleo del sistema y actúa como intermediaria entre la capa de presentación y la capa de datos. En ella se concentra toda la lógica de negocio que da sentido a los procesos de ParkNow. Sus responsabilidades abarcan:

- Recepción y procesamiento de solicitudes HTTP enviadas por el frontend.
- Validación de la información recibida.
- Aplicación de reglas de negocio, como el control de espacios disponibles o la generación de alertas cuando los cupos son limitados.

Para el envío y gestión de dichas alertas en tiempo real, el backend publica mensajes a un **broker MQTT**, al cual se suscriben los clientes interesados (por ejemplo, paneles de control o módulos de supervisión). Esto permite una comunicación asincrónica, eficiente y escalable entre los distintos componentes del sistema.

- Gestión de la autenticación y autorización de usuarios mediante JWT, asegurando que únicamente perfiles autorizados accedan a funcionalidades específicas.
- Coordinación de las operaciones de lectura y escritura sobre la base de datos.
Al centralizar la lógica en esta capa, se facilita la reutilización de funcionalidades y se asegura la coherencia en el procesamiento de las reglas del sistema.

- **Capa de Datos (Base de Datos):** Implementada en PostgreSQL, constituye el repositorio donde se almacena toda la información que da soporte al sistema. Su papel es fundamental, ya que garantiza la persistencia, integridad y consistencia de los datos. En esta capa se administran entidades críticas como:

- Usuarios y roles.
- Parqueaderos registrados.

- Vehículos ingresados.
- Espacios de estacionamiento y su estado (libre/ocupado).
- Sesiones de estacionamiento.
- Reportes generados por el sistema.

Esta estructura garantiza que la información sea consultada y actualizada de manera eficiente y confiable, proporcionando el soporte necesario para los procesos de negocio definidos en la capa de aplicación. En conjunto, las tres capas conforman una arquitectura lógica **sólida, modular y escalable**, que favorece el mantenimiento, la reutilización de componentes y la integración de futuras mejoras. De esta manera, el sistema **ParkNow** contará con una base tecnológica estable que le permitirá evolucionar de manera progresiva sin comprometer su rendimiento ni su confiabilidad.

3.2. APIs, Software y Sistemas de Información Involucrados

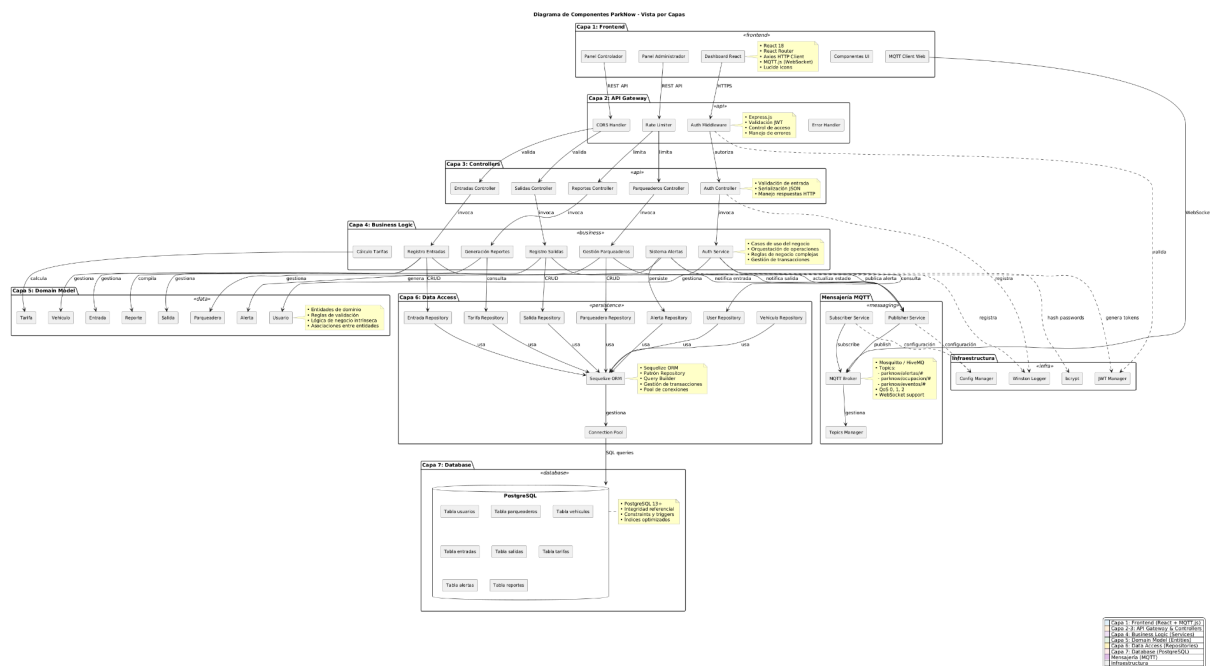
La arquitectura lógica de ParkNow se apoya en la integración de diferentes tecnologías y componentes de software que aseguran la comunicación entre capas y la ejecución de los procesos fundamentales del sistema.

- **API REST Principal:** Este componente es el encargado de gestionar la comunicación entre el frontend desarrollado en React y el backend implementado en Node.js. Se construye sobre Express.js e incorpora un conjunto de middlewares que permiten la autenticación y validación de datos. A través de esta API se exponen los endpoints responsables de la gestión de usuarios, parqueaderos, vehículos, espacios de estacionamiento y reportes, garantizando así la interacción centralizada y controlada entre los distintos módulos del sistema.
- **Sistema de Autenticación:** La seguridad en el acceso al sistema se implementa mediante un esquema de autenticación basado en JSON Web Tokens (JWT), lo que asegura que cada usuario pueda acceder únicamente a las funcionalidades asociadas a su rol. Para la protección de credenciales se utiliza bcrypt, encargado de generar el hash de las contraseñas. Este sistema de autenticación se encuentra embebido en el backend y actúa de manera transversal en todas las rutas que requieren validación, fortaleciendo la confiabilidad de la plataforma.
- **Sistema de Mensajería MQTT:** Se implementa un broker MQTT encargado de manejar la comunicación de eventos en tiempo real. El backend actúa como *publicador* de mensajes (por ejemplo, alertas de

cupos críticos), mientras que los clientes o paneles suscritos reciben dichas notificaciones al instante. Este mecanismo complementa el esquema HTTP/REST tradicional, aportando un canal liviano para eventos que requieren baja latencia y actualización continua.

- **Sistema de Gestión de Base de Datos:** La persistencia de los datos se gestiona a través de PostgreSQL, que actúa como repositorio central de la información del sistema. El acceso a la base de datos se realiza mediante Sequelize ORM, un mapeador objeto-relacional (ORM) que proporciona una capa de abstracción robusta sobre el motor de base de datos. Sequelize facilita las operaciones CRUD mediante modelos que representan las tablas de la base de datos, gestiona las relaciones entre entidades a través de asociaciones (hasMany, belongsTo, belongsToMany), y permite la ejecución de consultas complejas mediante su Query Builder integrado. La arquitectura de persistencia implementa el patrón Repository, separando la lógica de acceso a datos de la lógica de negocio. Cada entidad del dominio cuenta con su propio repositorio (UserRepository, ParqueaderoRepository, VehiculoRepository, etc.) que encapsula todas las operaciones de base de datos relacionadas con dicha entidad. Esta separación favorece la testabilidad del código y permite cambiar la implementación de persistencia sin afectar las capas superiores. Para optimizar el rendimiento en escenarios con múltiples usuarios concurrentes, Sequelize gestiona automáticamente un pool de conexiones que asegura un uso equilibrado de los recursos y una mayor disponibilidad en las operaciones de lectura y escritura. El pool reutiliza conexiones existentes en lugar de crear nuevas para cada petición, reduciendo significativamente la latencia y mejorando la escalabilidad del sistema.
- **Servidor Web / Proxy Reverso:** Como punto de entrada único al sistema se encuentra Nginx, configurado como proxy inverso. Su función principal es servir los archivos estáticos del frontend y redirigir las solicitudes hacia el backend en Node.js. De esta forma, se logra una arquitectura unificada que no solo mejora el rendimiento y la seguridad, sino que también ofrece a los usuarios una experiencia fluida y confiable en el acceso a las funcionalidades del sistema.

4. MODELO DE ARQUITECTURA LÓGICA



Para mejor visualización del diagrama visite el siguiente enlace:

 Componentes v2.pdf

Descripción de la Arquitectura Lógica

El sistema de información ParkNow adopta una arquitectura en capas (Layered Architecture) de 6 niveles, reconocida como una de las más robustas para el desarrollo de sistemas empresariales debido a su clara separación de responsabilidades, alta cohesión, bajo acoplamiento y facilidad de mantenimiento.

La definición de estas capas busca garantizar que los cambios en una de ellas no afecten directamente a las demás, permitiendo la evolución del sistema de manera modular y escalable.

- **Capa de Interfaz de Usuario (User Interface):** Esta capa representa la interfaz gráfica con la cual interactúan los usuarios finales. Está desarrollada completamente en React y se ejecuta en el navegador web del usuario. Entre sus principales responsabilidades se encuentran:
 - Renderizado de componentes visuales y manejo de la experiencia de usuario (UX).
 - Captura de eventos de usuario (clicks, formularios, navegación).
 - Presentación de datos recibidos desde las capas inferiores.
 - Enrutamiento del lado del cliente mediante React Router.
 - Gestión del estado local de la interfaz.

Esta capa se comunica exclusivamente con la capa de presentación mediante

peticiones HTTP/HTTPS a través de Axios, sin conocer los detalles de implementación de las capas inferiores.

- **Capa de Presentación (Presentation):** Implementada mediante controladores de Express.js, esta capa actúa como punto de entrada de las peticiones HTTP

provenientes del frontend. Sus responsabilidades incluyen:

- Recepción y enrutamiento de solicitudes HTTP.
- Validación inicial de parámetros de entrada.
- Invocación de casos de uso de la capa de aplicación.
- Serialización de respuestas en formato JSON.
- Manejo de códigos de estado HTTP apropiados.
- Gestión de errores y excepciones para respuestas al cliente.

Los controladores en esta capa están organizados por dominio funcional (AuthController, ParquaderosController, EntradasController, etc.) y delegan toda la lógica de negocio a la capa de aplicación.

- **Capa de Aplicación (Application):** Contiene los casos de uso del sistema y orquesta la lógica de negocio sin implementarla directamente. Esta capa coordina las operaciones entre la capa de presentación, el modelo de dominio

y la capa de persistencia. Sus responsabilidades principales son:

- Implementación de casos de uso específicos del negocio (registrar entrada, calcular tarifa, generar reporte, etc.).
- Coordinación de transacciones entre múltiples entidades.
- Aplicación de reglas de negocio complejas que involucran múltiples dominios.
- Gestión de la autenticación y autorización mediante JWT.
- Orquestación de operaciones entre repositorios.

Esta capa garantiza que la lógica de negocio permanezca independiente de los detalles de implementación de la persistencia y la presentación.

- **Capa de Modelo de Dominio (Domain Model):** Representa el corazón del sistema, conteniendo las entidades de negocio y las reglas de dominio fundamentales. En esta capa se definen:

- Entidades de negocio: Usuario, Parquadero, Vehículo, Entrada, Salida, Tarifa, Alerta, Reporte.
- Reglas de validación de dominio.
- Lógica de negocio intrínseca a cada entidad.
- Relaciones y asociaciones entre entidades.
- Cálculos de negocio (como cálculo de tarifas, disponibilidad de

espacios, generación de alertas).

Las entidades en esta capa son independientes de cualquier framework o tecnología de persistencia, lo que permite que el dominio sea portable y testeable de forma aislada.

- **Capa de Persistencia (Persistence):** Implementada mediante Sequelize ORM

y el patrón Repository, esta capa encapsula toda la lógica de acceso a datos. Sus responsabilidades incluyen:

- Definición de modelos Sequelize que mapean las entidades de dominio a tablas de base de datos.
- Implementación de repositorios que abstraen las operaciones CRUD.
- Gestión de relaciones entre modelos (asociaciones Sequelize).
- Construcción de consultas complejas mediante Query Builder.
- Manejo de transacciones de base de datos.
- Implementación de pool de conexiones para optimización de recursos.

El uso de Sequelize ORM proporciona una capa de abstracción que permite cambiar de motor de base de datos con mínimas modificaciones, además de

facilitar migraciones, validaciones y sincronización de esquemas.

- **Capa de Datos (Data):** Constituye el repositorio físico donde se almacena toda la información persistente del sistema. Implementada en PostgreSQL, esta capa garantiza:

- Persistencia permanente de datos.
- Integridad referencial mediante foreign keys y constraints.
- Consistencia transaccional mediante ACID.
- Optimización de consultas mediante índices estratégicos.
- Triggers para automatización de lógica de negocio crítica.
- Backup y recuperación ante desastres.

En esta capa se administran las tablas: usuarios, parqueaderos, vehiculos, entradas, salidas, tarifas, alertas y reportes.

Esta estructura de 6 capas garantiza que el sistema ParkNow cuente con una arquitectura robusta, mantenible y escalable. La separación de responsabilidades

permite que cada capa evolucione independientemente, facilitando el testing unitario, la reutilización de componentes y la incorporación de nuevas funcionalidades sin comprometer la estabilidad del sistema.

El flujo de comunicación entre capas es estrictamente unidireccional y

descendente: la capa de Interfaz de Usuario solo se comunica con Presentación, Presentación solo invoca a Aplicación, Aplicación coordina Dominio y Persistencia, y Persistencia accede a Datos. Esta disciplina arquitectónica previene dependencias circulares y mantiene la cohesión del diseño.

En esta arquitectura también se integra un mecanismo de mensajería en tiempo real basado en MQTT, que actúa de manera transversal entre la capa de Aplicación y las capas de Presentación o Interfaz de Usuario.

Este componente permite el envío inmediato de notificaciones y alertas del sistema sin requerir peticiones HTTP continuas, mejorando la eficiencia de comunicación entre el backend y los clientes conectados.

El broker MQTT se considera un servicio de soporte a la capa de Aplicación, que publica eventos relevantes (como alertas de cupos críticos) a los clientes suscritos en la interfaz de usuario.

5. ARQUITECTURA FÍSICA

5.1 Descripción de la Arquitectura Física

La arquitectura física de ParkNow está diseñada inicialmente para un entorno de sede única, con la posibilidad de escalar a múltiples sedes en futuras fases del proyecto. Esta configuración contempla un servidor principal encargado de alojar el backend, el servidor web y la base de datos; un conjunto de estaciones de trabajo utilizadas por los controladores para el acceso al sistema; y una infraestructura de red que conecta de manera segura y eficiente todos los dispositivos. Además, se incluye un sistema de respaldo energético que garantiza la disponibilidad continua de los servicios críticos.

5.2 Componentes de Hardware y Configuración

- **Servidor Principal (ParkNow-Server-01):**El sistema se despliega en un servidor físico o máquina virtual identificado con la dirección IP local 192.168.1.10, asignada por el router. Este nodo concentra los servicios principales: el backend desarrollado en Node.js, la base de datos en PostgreSQL y el servidor web Nginx configurado como proxy inverso. La comunicación con los clientes se realiza mediante los protocolos HTTP/HTTPS en los puertos 80 y 443, mientras que la administración remota se habilita a través de SSH. El sistema operativo recomendado para este servidor es Ubuntu Server 20.04 LTS, dada su estabilidad y soporte de seguridad.
- **Estaciones de Trabajo de Controladores:**Se consideran tres estaciones de trabajo principales (ParkNow-WS-01, ParkNow-WS-02 y

ParkNow-WS-03), con direcciones IP asignadas dinámicamente mediante DHCP en el rango 192.168.1.100-150. Cada estación cuenta con un navegador web moderno que permite el acceso a la aplicación cliente a través de HTTPS. Los sistemas operativos sugeridos para estas estaciones son Windows 10/11 o Ubuntu Desktop, garantizando compatibilidad y facilidad de uso para el personal.

- **Sistema de Alimentación Ininterrumpida (UPS):** Para garantizar la continuidad de los servicios ante posibles fallos eléctricos, se incorpora un sistema UPS de 1500VA, al cual se conectan los dispositivos críticos, incluyendo el servidor principal y el router. Este sistema puede contar adicionalmente con monitoreo remoto mediante SNMP, facilitando la supervisión del estado de la alimentación.
- **Router/Switch Principal:**
La infraestructura de red está soportada por un router WiFi empresarial configurado con la dirección IP **192.168.1.1**, que actúa como *gateway* y punto de enlace entre la red interna y la conexión a Internet. Este equipo proporciona servicios de **DHCP, NAT y firewall básico**, asegurando tanto la conectividad como la seguridad. Además, su función de switch permite la interconexión de todos los dispositivos de la red local.
- **Conexión a Internet:**
El sistema requiere de una conexión de al menos **50 Mbps de ancho de banda**, suficiente para el acceso concurrente de controladores y la administración remota del servidor. La conexión se canaliza a través del router principal, que distribuye el tráfico a las estaciones de trabajo y al servidor.

5.3 Protocolos de Comunicación

La comunicación entre los diferentes componentes del sistema se establece a través de los siguientes protocolos:

HTTP/HTTPS: Canal principal de interacción entre las estaciones de trabajo y el servidor web.

TCP/IP: Protocolo base para la comunicación de red entre todos los dispositivos.

SSH (puerto 22): Protocolo seguro de administración remota para el servidor principal.

SQL sobre TCP (puerto 5432): Comunicación entre el backend y la base de datos PostgreSQL.

MQTT (puerto 1883 o 8883): Protocolo de mensajería ligera utilizado para la transmisión en tiempo real de alertas y eventos del sistema entre el servidor y los clientes suscritos.

5.4 Servicios a Implementar

En el servidor principal se desplegarán los siguientes servicios:

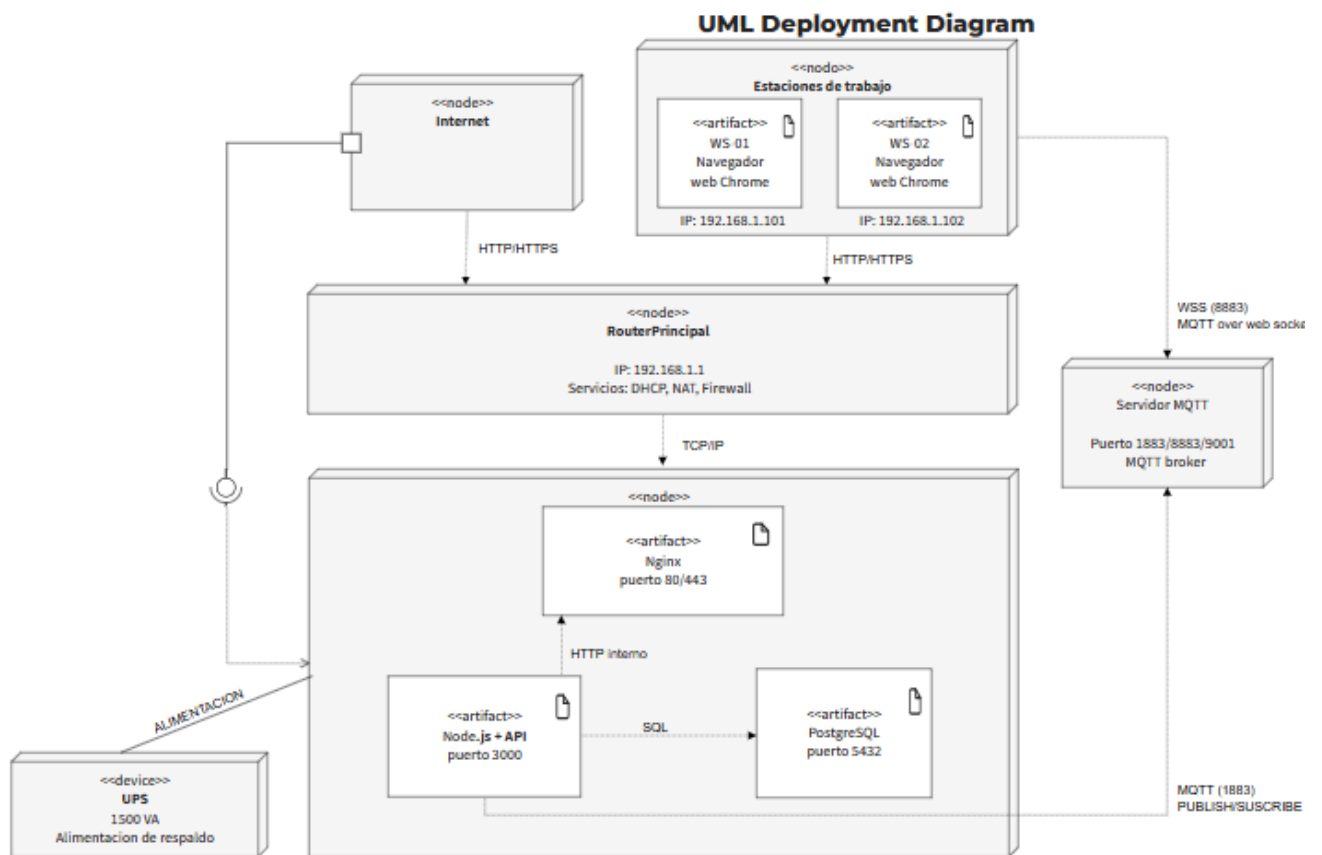
Servicio web Node.js en el puerto 3000, encargado de la lógica de negocio.

Servicio de base de datos PostgreSQL en el puerto 5432, para la persistencia de la información.

Servicio de proxy Nginx, configurado en los puertos 80 y 443 para atender peticiones HTTP/HTTPS.

Servicio de mensajería MQTT, encargado de gestionar la publicación y suscripción de eventos de alerta en tiempo real. Este servicio se ejecuta junto al backend en el servidor principal, comunicándose mediante el puerto 1883 o 8883 según la configuración de seguridad.

6. MODELO DE ARQUITECTURA FISICA



El diagrama de despliegue presentado corresponde a la arquitectura física del sistema de información ParkNow. Este modelo muestra los nodos de hardware, los artefactos de software desplegados en cada uno de ellos y los protocolos de comunicación empleados para garantizar el funcionamiento del sistema.

En primer lugar, se encuentra el nodo **Internet**, que representa el medio a través del cual los usuarios acceden al sistema utilizando navegadores web modernos. La comunicación se realiza mediante los protocolos **HTTP/HTTPS**, asegurando un acceso confiable y seguro.

La conexión hacia la red interna es gestionada por el nodo **Router Principal**, configurado con la dirección IP **192.168.1.1**. Este dispositivo ofrece servicios de **DHCP, NAT y firewall**, permitiendo la asignación de direcciones IP y protegiendo la red interna frente a accesos no autorizados. A través de **TCP/IP**, el router direcciona el tráfico hacia los demás componentes de la infraestructura.

El núcleo de la solución se encuentra en el **Servidor Principal**, donde se despliegan los siguientes artefactos de software:

- **Nginx (puertos 80/443):** Actúa como servidor web y proxy inverso, recibiendo las solicitudes HTTP/HTTPS desde los navegadores y rediriéndolas al backend desarrollado en Node.js.
- **Node.js + API (puerto 3000):** Implementa la lógica de negocio del sistema y procesa las peticiones provenientes de Nginx, además de gestionar operaciones de registro, autenticación y control de espacios.
- **PostgreSQL (puerto 5432):** Motor de base de datos relacional que almacena la información de usuarios, vehículos, parqueaderos y reportes. Se comunica con la API mediante consultas SQL.

Como soporte a la operación, se incluye un nodo **UPS (1500 VA)** que proporciona alimentación de respaldo en caso de fallas eléctricas, garantizando la continuidad del servicio en el servidor y los dispositivos de red.

Adicionalmente, se representan las **Estaciones de Trabajo**, correspondientes a los equipos de los controladores del sistema. Cada estación cuenta con su dirección IP asignada (por ejemplo, **192.168.1.101** y **192.168.1.102**) y accede al sistema mediante un **navegador web (Chrome)**, comunicándose con el servidor a través del protocolo **HTTP/HTTPS**.

Finalmente, los protocolos de comunicación que articulan la arquitectura son:

- **HTTP/HTTPS (80/443):** Comunicación entre estaciones de trabajo e interfaz web (Nginx).
- **HTTP interno (3000):** Comunicación entre Nginx y el backend (Node.js).
- **SQL (5432):** Comunicación entre el backend y la base de datos PostgreSQL.
- **TCP/IP:** Protocolo de transporte de la red local, que enlaza todos los componentes.